

Understanding Energy Consumption of Disaggregated and Non-Disaggregated LLM Serving on Datacenter and Edge GPUs

Tian Liu

Abstract—Abstract.

Index Terms—Large language model serving, energy efficiency, disaggregated inference, GPU, performance measurement.

1 INTRODUCTION

Large language models (LLMs) have become a general-purpose computing substrate for conversational assistants, code generation, retrieval-augmented generation, document and scientific-literature analysis, automated scientific discovery, and increasingly autonomous multi-agent workflows [1]–[6]. These applications transform pretrained models into continuously accessed online services rather than one-time offline computations. A production service must accept requests with heterogeneous prompt and response lengths, maintain interactive latency under changing concurrency, and support models whose parameters and runtime states span multiple accelerators. As models acquire longer contexts and applications compose multiple model calls, inference efficiency has therefore become as important as model capability.

To support this demand, systems research has optimized LLM inference along several complementary dimensions. Scheduling and continuous batching improve accelerator utilization and balance throughput against latency [7]–[11]. KV-cache research reduces fragmentation, reuses shared prefixes, separates cache storage from computation, or selectively retains important states to control the memory cost of long-context generation [12]–[21]. Multi-GPU parallelism partitions model tensors, routed experts, or long sequences to overcome the compute and memory limits of a single device [22]–[26]. Disaggregated architectures further separate inference phases or operators with different bottlenecks so that they can be provisioned independently [27]–[31]. Together, these advances improve throughput, latency, memory capacity, and scalability, making increasingly large and complex inference services practical. Their performance benefits, however, do not directly establish their energy efficiency.

The rapid expansion and continuous operation of LLM services create a growing energy challenge. Online inference repeatedly streams large weight tensors, reads and writes expanding KV caches, exchanges intermediate state across GPUs, and keeps provisioned accelerators powered while traffic fluctuates. Small inefficiencies at the request level accumulate across millions of invocations and long service lifetimes. The International Energy Agency estimates that data centers consumed about 415 TWh in 2024, approximately 1.5% of global electricity use, and projects consumption to reach roughly 945 TWh by 2030, with AI as the most important driver of this growth [32]. Energy is thus becoming a first-order constraint on operating cost, power provisioning, and sustainable growth. Importantly, lowering instantaneous power does not guarantee lower energy: a slower operating point may extend request execution, and an architecture that improves utilization may introduce communication, synchronization, or idle-device overhead [33], [34].

Hardware diversity further complicates this trade-off. GPUs differ substantially in arithmetic throughput, memory capacity, memory bandwidth, interconnect support, and power-management interfaces. A compute-rich GPU can accelerate prefill or FFN computation but remain inefficient for memory-bound decoding if its capacity or bandwidth is insufficient; conversely, a memory-rich datacenter GPU can support large models and KV caches even when its peak arithmetic throughput is lower. Datacenter GPUs generally emphasize ECC memory, sustained operation, manageability, isolation, and scale-up/scale-out communication, whereas consumer GPUs prioritize high local throughput and cost but usually provide less memory and fewer datacenter interconnect and management features [35]–[38]. Frequency controls also differ across products rather than following a universal class rule. On our platforms, the A800 exposes practical runtime control of the SM/core clock while its memory clock remains fixed, whereas the RTX 5090 platform also permits memory-clock adjustment. These differences can change not only absolute performance and energy, but also which serving architecture and role placement are preferable.

Existing energy research can be grouped into three broad directions. Measurement studies quantify request-, token-, workflow-, or component-level energy and show

• Tian Liu is with the University of Science and Technology of China, Hefei, Anhui, China. E-mail: ltian@mail.ustc.edu.cn.

that workload shape, batching, model structure, and hardware all matter [34], [39], [40]. Runtime-management systems adapt resource allocation, model parallelism, batching, power caps, or GPU frequency under service-level objectives [41]–[45]. Phase- and operator-aware systems exploit the asymmetric behavior of prefill, decode, attention, and FFN computation through finer-grained placement and frequency control [46]–[48]. These studies establish important mechanisms and observations, but optimization systems generally target a particular architecture or policy, while measurement studies do not systematically compare alternative disaggregation boundaries. We still lack a controlled, architecture-level account of how different disaggregated designs behave across deployment plans, hardware platforms, communication environments, and runtime configurations. Without such an account, it remains unclear when specialization yields genuine energy savings and when it merely shifts cost among devices or converts computation into communication and idle time.

We address this gap through a progressive measurement study of non-disaggregated (Native), prefill–decode-disaggregated (PD), and attention–FFN-disaggregated (AF) serving in a common runtime. We begin with a broad characterization across dense and MoE models and representative input/output regimes to establish the performance–energy landscape and identify an architecture-sensitive MoE setting. We then examine how communication environments change the relative behavior of the architectures, followed by a study of data, tensor, expert, and hybrid parallel deployments. Finally, we move from software configuration to hardware characteristics, comparing GPUs with different compute and memory capabilities and exploring heterogeneous assignments in which specialized roles use different devices. This progression moves from application-visible variation to communication, deployment, and hardware causes, while matched traces, SLO-aware filtering, repeated open-loop runs, and deployment-level energy accounting keep comparisons controlled.

Our study makes the following contributions:

- **An architecture-centered energy characterization.** We formulate and implement a common measurement boundary for Native, PD, and AF, enabling direct comparison of complete deployments rather than isolated kernels or worker pools. This exposes the fundamental trade-off between resource specialization and the transfer, synchronization, and device-residency costs introduced by disaggregation.
- **Evidence that architecture choice is workload dependent.** Completed repeated RDMA measurements exhibit a crossover: PD consumes less GPU energy for short QA and decode-heavy chatbot traffic, whereas AF consumes less for prefill-heavy RAG, long-output summarization, and long-context traffic; the two are nearly tied on the balanced workload. Thus, neither form of disaggregation is uniformly preferable, and input/output structure must be part of an energy-aware architecture decision.
- **A cross-layer explanation of disaggregated-serving energy.** By progressively controlling network conditions and parallel placement, our study separ-

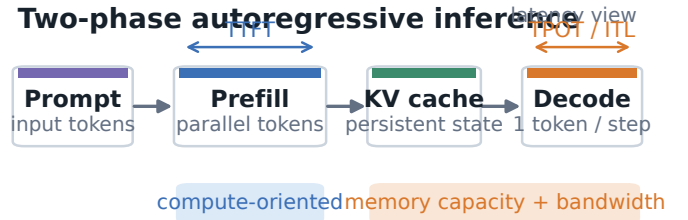


Fig. 1. The prefill and autoregressive decode phases of LLM inference.

rates useful phase/operator specialization from state transfer, collective communication, pipeline bubbles, and idle-power overhead. This analysis connects request-level energy outcomes to their underlying mechanisms rather than reporting aggregate joules alone.

- **A hardware-aware path toward heterogeneous serving.** We relate architecture behavior to GPU compute throughput, memory capacity, bandwidth, interconnect, and controllable frequency domains. The resulting analysis provides an empirical basis for assigning different GPU classes to specialized roles and for future architecture- and hardware-aware controllers.

The remainder of this paper is organized as follows. Section 2 introduces LLM inference, parallel deployment, disaggregated serving, energy management, and GPU classes. Section 3 reviews related work and positions our measurement scope. Section 4 describes the hardware, models, workloads, architectures, and measurement methodology. Section 5 presents the experimental questions and results. Section 6 discusses their implications, limitations, and future directions, and Section 7 concludes.

2 BACKGROUND

2.1 LLM Inference

Autoregressive LLM inference has two execution phases (Figure 1) [12], [27], [29], [30]. During *prefill*, the model processes all prompt tokens in parallel and materializes their key–value (KV) states. Prefill typically exposes substantial matrix-multiplication parallelism and is therefore compute intensive. During *decode*, the model generates one token per iteration while reading the weights and an expanding KV cache. Decode consequently has lower arithmetic intensity and is often constrained by memory capacity and bandwidth. Time to first token (TTFT) captures prompt-processing and queueing delay, whereas time per output token (TPOT) and inter-token latency (ITL) capture decode responsiveness. Input/output lengths, batching, and offered load change the balance between these phases and thus their performance and energy behavior.

2.2 Parallel Deployment of LLMs

Large models and high request rates require multiple GPUs. Figure 2 illustrates the three parallel dimensions considered in this work [22], [23], [25], [26], [33]. Data parallelism (DP) replicates the model and partitions requests among replicas,

Parallel deployment dimensions

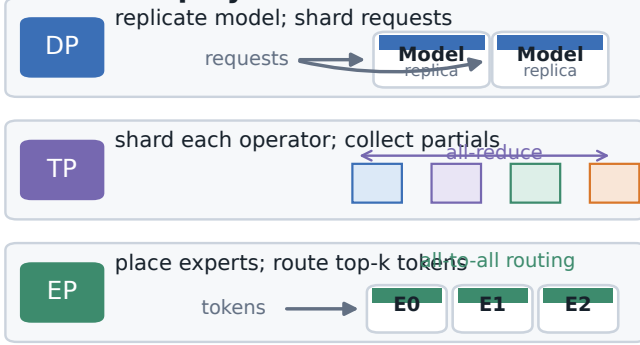


Fig. 2. DP replicates models, TP shards operators, and EP distributes routed experts.

increasing aggregate throughput at the cost of replicated weights. Tensor parallelism (TP) shards tensors or operators across GPUs and uses collectives to combine partial results; it reduces per-GPU memory demand but adds communication to each layer. Expert parallelism (EP) distributes MoE experts across GPUs and uses token dispatch and combine communication around routed FFN computation. Real deployments often combine DP, TP, and EP. Their compute utilization, memory footprint, and communication volume differ, so the fastest or lowest-power plan is not necessarily the most energy-efficient one.

2.3 Disaggregated Serving Architectures

A conventional Native deployment executes prefill, decode, attention, and FFN computation in the same worker group. Prior systems introduced PD for phase-specialized serving and AF for disaggregated expert execution [27]–[31]. Disaggregation separates components with different resource characteristics so that they can be provisioned and scaled independently (Figure 3). Prefill–decode disaggregation (PD) assigns the two inference phases to separate pools and transfers request state and KV data from prefill workers to decode workers. Attention–FFN disaggregation (AF) separates the attention/KV path from dense or expert FFN computation and exchanges activations at transformer-layer boundaries. PD can match compute-oriented prefill with memory-oriented decode resources, while AF can specialize resources for attention and FFN. Both, however, introduce communication, synchronization, and pipeline overhead; whether separation saves energy depends on workload shape, parallel placement, and the underlying interconnect.

2.4 Energy Management for LLM Serving

Energy is the integral of power over execution time. A lower power draw therefore does not guarantee lower energy if it also extends request processing. Modern GPUs expose several control mechanisms, including core and memory dynamic voltage and frequency scaling (DVFS), power caps, and device consolidation [40]–[42], [47], [49]. Serving systems additionally control batching, placement, parallelism, and replica count. As shown in Figure 4, an energy manager

Where the execution boundary is placed

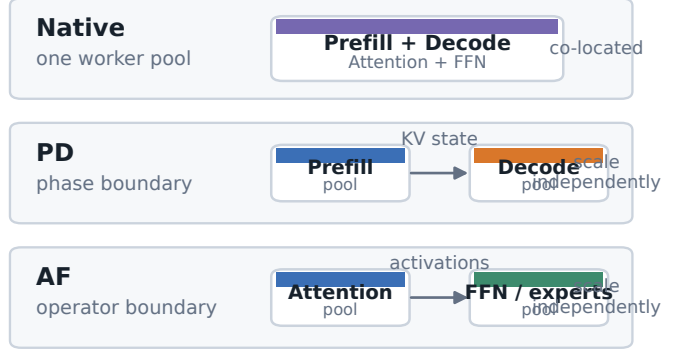


Fig. 3. Native, prefill–decode (PD), and attention–FFN (AF) serving architectures.

SLO-constrained energy control loop

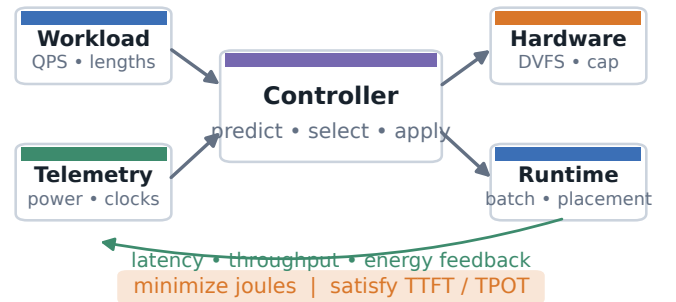


Fig. 4. Closed-loop energy management for SLO-constrained LLM serving.

observes workload and system state, selects hardware and serving controls, and receives feedback through latency, throughput, power, and energy measurements. Useful policies must preserve TTFT/TPOT objectives while minimizing joules per request or maximizing tokens per joule. The best setting is architecture dependent because Native, PD, and AF expose different compute slack, memory traffic, and communication stalls.

2.5 Datacenter and Consumer GPUs

GPUs used for LLM inference broadly fall into datacenter and consumer classes. Datacenter GPUs are designed for sustained multi-user operation and typically emphasize large error-correcting memory, predictable thermal behavior, remote manageability, virtualization, and high-speed scale-up or scale-out communication. Depending on the product family, they may provide NVLink/NVSwitch, GPUDirect RDMA, Multi-Instance GPU support, and stable power or clock controls. These features make datacenter GPUs suitable for multi-GPU serving and for deployments in which communication, availability, and isolation are first-order concerns [35], [36], [38].

Consumer GPUs target interactive graphics and workstation workloads. They can offer high arithmetic throughput and favorable acquisition cost, but generally provide

less memory, lack datacenter features such as NVLink and MIG, and have different cooling, reliability, and management assumptions. For example, the RTX 5090 exposes high Tensor throughput and fast GDDR7 memory but is a PCIe device without NVLink [37]. Consequently, peak FLOPS alone do not determine suitability for LLM serving: model capacity, memory behavior, communication requirements, power limits, and operational constraints all affect performance and energy efficiency. This distinction motivates evaluating both datacenter and consumer GPU classes rather than extrapolating conclusions from a single platform.

3 RELATED WORK

LLM serving and resource management. Modern serving systems improve throughput and latency through iteration-level scheduling, continuous or hybrid batching, request migration, and SLO-aware resource allocation [7]–[12]. Constraint-aware schedulers select batch sizes and parallel configurations from workload distributions, while simulation frameworks use operator profiles to explore large deployment spaces [50]. Recent systems further adapt model sharding across prefill and decode or optimize placement over heterogeneous GPUs and networks [26], [44], [45]. These systems primarily optimize goodput, latency, utilization, or monetary cost; energy is rarely the principal comparison boundary.

Disaggregated inference. P/D systems isolate compute-intensive prefill from memory-intensive decode so that each phase can use an independent parallel plan and resource pool. DistServe co-optimizes phase placement and goodput under TTFT/TPOT objectives, while Splitwise studies homogeneous and heterogeneous phase-specific clusters [27], [29]. DejaVu streams KV state to support disaggregation and fault tolerance, and Mooncake builds a KVCache-centric transfer and storage substrate [14], [30]. Other systems combine disaggregation with chunked prefill, dynamic re-sharding, or data-reduced orchestration to reduce phase interference and transfer pressure [26]. Beyond P/D, InfiniteLLM separates attention and non-attention execution for distributed long-context serving, MegaScale-Infer disaggregates attention and experts, and OPSCALE provisions individual operators [28], [31], [51]. This literature establishes that the disaggregation boundary changes utilization and communication, but does not systematically compare the resulting energy behavior across Native, PD, and AF.

KV-cache and long-context optimization. KV-cache work spans paging, compression, sparsity, offloading, streaming, and reuse. PagedAttention reduces fragmentation, RadixAttention reuses shared prefixes, Marconi manages prefix state for hybrid models, and CacheBlend selectively recomputes reusable caches for RAG contexts [12], [13], [52], [53]. Quantization and selective-attention methods reduce cache bytes or accessed pages, while unified sparse attention addresses both long-prefill computation and decode KV traffic [15]–[18], [21], [54], [55]. Offloading systems prefetch selected KV entries from CPU memory or exploit memory elsewhere in a scale-up GPU domain [19], [56]. CacheGen compresses KV state for network streaming, making it directly relevant to disaggregated transfer cost [20]. Long-context systems

additionally use elastic sequence parallelism and customizable attention kernels to balance computation, memory, and communication [25], [57]. These methods alter the amount and location of state moved by PD and AF, yet their deployment-level energy consequences remain under-explored.

Power management and energy-efficient serving. A growing body of work coordinates runtime decisions with GPU power controls. DynamoLLM profiles model and request-class energy behavior, maintains differently configured instance pools, and uses hierarchical controllers to resize pools and select tensor-parallel and GPU-frequency configurations under latency SLOs [41]. throttLL’eM predicts future batch and KV-cache states and selects an energy-efficient frequency and engine configuration [42]. GreenLLM applies phase-aware frequency scaling, BiScale combines P/D placement with fine-grained DVFS, and PALS jointly controls GPU power caps and batch size for MoE serving [43], [46], [47]. AFlex combines P/D and A/F disaggregation into independently provisioned PA, PF, DA, and DF pools with operator-phase-specific frequency control [48]. Fine-grained accelerator models and recent inference-energy benchmarks broaden the available control and measurement methodology [40], [49]. These approaches optimize particular architectures or knobs; our goal is to characterize how the architecture itself changes the available energy trade-off.

Characterizing performance and energy. Measurement work shows that power, runtime, and energy must be evaluated together. Distributed-training studies demonstrate that parallel configuration and GPU frequency interact through different computation-communication ratios [33]. For inference, recent studies characterize multi-request workflows, request- and token-level costs, and cross-model or cross-hardware behavior [34], [39], [40]. Hardware benchmarks likewise expose substantial performance variation across GPU and accelerator families [50]. However, these studies focus on training, monolithic inference, application workflows, or broad benchmark coverage. They do not isolate the energy impact of alternative disaggregation boundaries under matched workloads, networks, parallel plans, and GPU characteristics.

Comparison with existing work. Table 1 summarizes the scope of representative studies. Existing measurement work mainly considers monolithic inference or distributed training, whereas disaggregated-serving work primarily targets performance and resource efficiency. Energy-aware serving systems optimize a particular architecture or control mechanism. In contrast, our study uses a common measurement methodology to compare Native, PD, and AF serving across multiple GPU classes, interconnects, and parallel deployment paradigms.

4 EXPERIMENTAL SETUP AND METHODOLOGY

This section describes the experimental platforms, workloads, deployment configurations, and measurement methodology. The individual experiments and research questions are presented in Section 5.

4.1 Hardware Configuration

Our hardware matrix covers both datacenter and consumer GPUs with different compute throughput, memory capacity,

TABLE 1
Scope comparison with representative existing work. “Energy” denotes direct energy measurement or optimization.

Work	Energy	Native	PD	AF	Multiple GPU classes	Network effects	DP/TP/EP
DistServe [27]	–	–	Yes	–	–	–	–
MegaScale-Infer [28]	–	–	–	Yes	Yes	Yes	Yes
DynamoLLM [41]	Yes	Yes	–	–	–	–	TP
BiScale [47]	Yes	–	Yes	–	–	Yes	–
throttLL’eM [42]	Yes	Yes	–	–	Yes	–	TP
Req-Token Energy [39]	Yes	Yes	–	–	Yes	–	–
Zhang et al. [33]	Yes	–	–	–	Yes	Yes	Yes
This work	Yes	Yes	Yes	Yes	Yes	Yes	Yes

TABLE 2
Principal GPU platforms. Compute values are dense peak TFLOPS without structured sparsity.

GPU	FP32	FP16/BF16	Memory	Bandwidth	Class
A800 SXM4	19.5	312	80 GB HBM2e	2039 GB/s	Datacenter
L40S	91.6	362	48 GB GDDR6	864 GB/s	Datacenter
RTX 5090	104.8	419	32 GB GDDR7	1792 GB/s	Consumer
L20	59.3	119.5	48 GB GDDR6	864 GB/s	Datacenter

memory technology, and interconnect support. Table 2 summarizes the principal devices. NVIDIA A800-SXM4-80GB servers form the primary datacenter platform. Each server contains eight GPUs connected by NVLink/NVSwitch; PCIe and GPUDirect RDMA are used for intra-host and inter-host communication experiments, respectively. L40S provides a datacenter comparison with higher arithmetic throughput but lower memory capacity and bandwidth. RTX 5090 is the consumer-GPU platform, while L20 is a PCIe datacenter GPU. Both use GDDR memory and lack NVLink, complementing the tightly coupled A800 SXM platform.

To isolate network effects, only the A800 platform is used for the network comparison. A800 deployments exercise NVLink/NVSwitch within a server, PCIe-isolated placements, and GPUDirect RDMA across two or four servers while preserving the intended GPU budget. L40S, RTX 5090, and L20 are not included in the network-variable experiment; they use their platform-local PCIe connectivity. Every run records the GPU model and UUID, participating devices, GPU–NIC affinity, applied clock state, and observed transport so that an unintended communication path cannot be mistaken for the target topology.

Host CPUs launch workers, route requests, and coordinate measurements, but CPU type and CPU parallelism are not experimental variables. Paired comparisons use the same host allocation and process placement. Because the current energy scope is the set of GPUs assigned to a deployment, CPU and network-device energy are not included in the reported joules; this boundary is stated explicitly when interpreting cluster-level energy results.

4.2 Models, Workloads, and Parallelism

We select dense and mixture-of-experts (MoE) models at three scales (Table 3). For dense models, Small denotes fewer than 10 billion parameters, Middle denotes 10–50 billion, and Large denotes more than 50 billion. MoE systems have substantially different total and active parameter

counts, so we use deployment-oriented tiers rather than forcing dense-model thresholds onto them: OLMoE-1B-7B represents Small, Qwen3-30B-A3B represents Middle, and Mixtral-8×7B represents Large. Total parameters capture memory and placement cost, whereas active parameters approximate per-token expert computation.

The workload suite contains six fixed input/output shapes (Table 4). QA represents short-prefill/short-decode traffic, Chatbot is decode-heavy, Balanced has moderate input and output lengths, RAG is prefill-heavy, Summary combines a long prompt with a long response, and Long-Context stresses long-prefill execution and KV-cache capacity. Requests are generated once from fixed seeds and replayed across configurations. An open-loop generator issues requests according to a Poisson process, preventing server response time from altering the offered arrival rate.

We support data parallelism (DP), tensor parallelism (TP), expert parallelism (EP), and their hybrid combinations. DP creates complete model replicas and distributes requests among them. TP shards dense operators across GPUs and introduces collective communication. EP distributes MoE experts and routes tokens to their selected experts. A deployment manifest records the DP, TP, and EP degree independently for every role. Unless parallelism itself is varied, a configuration uses the same total GPU budget as its architectural counterparts.

4.3 Serving Architectures

We implement three serving architectures in SGLang. *Native* is the non-disaggregated baseline: one worker group executes the complete model and colocates prefill and decode as well as attention and FFN computation. Its default eight-GPU configuration is TP8.

PD separates prefill and decode into independent worker groups. The prefill group computes the prompt and transfers the resulting request state and KV cache to the decode group. Its default eight-GPU configuration assigns four GPUs to each group, denoted P-TP4 + D-TP4. The groups can also be placed on separate hosts to exercise RDMA communication.

AF separates attention and FFN execution. Attention workers maintain the attention/KV path, while FFN workers execute dense or routed-expert computation; intermediate activations cross the pool boundary at transformer-layer granularity. Its default eight-GPU configuration is A-TP4 + F-TP4. AF can use NVLink within a host or RDMA across hosts, and each pool can independently use DP, TP, or EP.

TABLE 3
Models included in the measurement matrix.

Scale	Dense	MoE
Small	Llama-3.1-8B	OLMoE-1B-7B
Middle	Qwen3-32B	Qwen3-30B-A3B
Large	Llama-3.3-70B	Mixtral-8×7B

All comparisons hold model precision, serving software, request stream, and total GPU allocation constant unless the evaluated configuration explicitly changes one of them. Prefix reuse is disabled to prevent cache-hit variation from confounding architecture comparisons. The complete deployment, rather than an individual worker or pool, is the unit of performance and energy accounting.

4.4 Measurement Methodology and Metrics

Each configuration is launched as a fresh deployment and must pass readiness and health checks before measurement. After any required warm-up, the load generator replays the frozen request stream and waits for all admitted requests to complete or time out. Configurations are executed serially to avoid resource interference, and each reported point uses three independent repetitions. The deployment manifest and system snapshots preserve model, placement, topology, process, and clock information needed to reproduce a run.

We obtain GPU energy from NVML cumulative-energy counters and calculate consumed energy from counter differences over the same interval used for request measurement. Cluster GPU energy is the sum over only the GPUs assigned in the deployment plan, and average GPU power is total GPU energy divided by measurement duration. Per-GPU and per-node values are retained to reveal role or placement imbalance. On devices or clock domains that do not expose a reliable cumulative counter, the applicable telemetry source and integration method are recorded with the result.

Performance metrics include offered and achieved QPS, input- and output-token throughput, end-to-end latency, time to first token (TTFT), time per output token (TPOT), and inter-token latency (ITL). We report mean and p50/p90/p95/p99 latency where applicable. Energy-efficiency metrics include total joules, average power, joules per successful request, joules per input token, joules per output token, joules per total token, and input/output/total tokens per joule. Network configurations additionally record link validation and telemetry; system snapshots include GPU utilization, memory use, temperature, and core/memory clocks.

A run is valid only when the deployment remains healthy, at least 99% of requests succeed, and the load generator reproduces the configured open-loop arrival process. We compare energy efficiency only among configurations satisfying the same service-level objective and having all three repetitions. This joint treatment of runtime, power, and energy is necessary because reducing instantaneous power

TABLE 4
Fixed-shape workloads; lengths are in tokens.

Workload	Input	Output	Regime
QA	128	64	LPLD
Chatbot	128	1024	LPHD
Balanced	512	256	MPMD
RAG	4096	64	HPLD
Summary	4096	1024	HPHD
LongContext	16384	256	Long context

can increase execution time and therefore fail to reduce total energy [33].

5 RESULTS

This section presents the evaluation results by answering the following research questions.

RQ1: Which dataset is more energy efficient for which deployment architecture?

RQ2: Which communication efficiency is more beneficial to which discrete architectures?

RQ3: How different parallel deployment paradigms affect different decoupling architectures?

RQ4: Which compute-memory ratio offers the best prospects for energy efficiency in which discrete architecture?

RQ5: Investigating the impact of HBM frequency tuning on different discrete architectures?

5.1 RQ1

5.2 RQ2

5.3 RQ3

5.4 RQ4

5.5 RQ5

6 DISCUSSION

6.1 Interpreting Performance–Energy Trade-offs

The energy efficiency of an LLM serving architecture is determined jointly by average power and execution time. A deployment that lowers GPU utilization or operating frequency may draw less power but consume more total energy if communication, synchronization, or slower computation substantially extends the measurement interval. Conversely, a high-power configuration can be energy efficient when it completes requests quickly and returns accelerators to an idle or reusable state. Therefore, architecture comparisons should not rank systems by power, latency, or throughput in isolation. We interpret a configuration as preferable only when it satisfies the same service-level objective (SLO) and improves normalized energy metrics such as joules per successful request or tokens per joule.

This distinction is particularly important for disaggregated serving. PD and AF can specialize worker pools for phases or operators with different resource demands, but they also introduce data movement and coordination. The relevant question is not whether disaggregation reduces the power of an individual pool, but whether the reduction in computation or interference outweighs transfer energy, synchronization delay, pipeline bubbles, and the baseline

power of additional active GPUs. Reporting per-role energy alongside deployment-level energy helps identify whether an apparent gain comes from genuine efficiency or from shifting work to another pool.

6.2 Implications for Deployment Design

No serving architecture should be expected to dominate across all workloads and platforms. Prefill-heavy workloads expose more compute parallelism, whereas decode-heavy and long-context workloads place greater pressure on model and KV-cache bandwidth. Native serving avoids inter-pool transfers but couples components with different scaling behavior. PD can independently provision compute-oriented prefill and memory-oriented decode, while AF can specialize attention and FFN resources, particularly for MoE models. The preferred architecture can consequently change with the input/output ratio, offered load, model type, and SLO.

Communication and parallelism must be considered as part of the architecture rather than as independent implementation details. TP and EP add frequent collectives or token exchange; PD transfers request state and KV data; and AF exchanges activations at layer boundaries. Fast scale-up links can hide or amortize some of these costs, whereas PCIe or multi-node RDMA may move the energy–performance crossover. Similarly, increasing DP can raise throughput through replication but also duplicates model memory and idle power. Practical provisioning should therefore select the architecture, role placement, and DP/TP/EP degrees jointly.

Hardware selection creates another trade-off. Datacenter GPUs provide large ECC memory and high-speed interconnects, while consumer GPUs can provide high arithmetic throughput at different capacity, management, and connectivity points. Peak FP16/BF16 throughput alone does not predict serving efficiency: decode can remain memory-bound, a model may not fit at the desired parallel degree, and a disaggregated design may depend more on communication than arithmetic capability. Frequency controls should likewise be applied selectively. Compute-bound components are more sensitive to core-frequency reductions, whereas memory- or communication-stalled components may tolerate them. Memory-frequency reductions are useful only when their power savings exceed the additional service time and do not violate the SLO.

6.3 Measurement Scope and Threats to Validity

Our conclusions are bounded by the evaluated models, request shapes, hardware, software version, and deployment configurations. The fixed-shape workloads isolate input/output effects and improve repeatability, but production traffic includes correlations, multi-turn context, shared prefixes, burstiness, cancellations, and changing output distributions. Open-loop Poisson arrivals do not reproduce every production process. Results should therefore be interpreted as controlled architectural characterization rather than a complete model of every deployment.

Hardware and software effects also limit generalization. GPU firmware, drivers, kernels, quantization, attention backends, routing policies, and batching algorithms can

change utilization and energy. Product-level peak specifications do not guarantee realized throughput. We mitigate these effects by keeping the software and precision fixed within paired comparisons, recording manifests and system state, validating communication paths, and repeating deployments; nevertheless, results from the evaluated GPUs should not be directly extrapolated to accelerators with different memory hierarchies or interconnects.

The energy boundary is another limitation. Our primary metric includes the GPUs assigned to a deployment because their cumulative energy counters provide a consistent measurement across configurations. It does not include host CPU, DRAM, NIC, switch, storage, cooling, or power-conversion losses. This boundary is suitable for comparing GPU execution but can understate the cost of multi-node and communication-intensive architectures. Future work should combine device counters with node- and rack-level metering and should account for embodied carbon and regional electricity carbon intensity.

Finally, SLO filtering can introduce survivorship bias if only successful high-load configurations remain. We therefore report offered and achieved load, success rate, and latency distributions and compare energy only among configurations satisfying the same validity criteria. Repeated trials quantify run-to-run variation, but longer-duration studies are still needed to capture thermal drift, failures, autoscaling transients, and diurnal workloads.

6.4 Toward Architecture-Aware Energy Management

The measurement dimensions in this study suggest that energy management should be architecture aware. Rather than tuning frequency, batch size, parallelism, or placement independently, a controller could use workload composition and measured phase/operator sensitivity to choose a complete operating point. Such a controller would select among Native, PD, and AF; assign hardware to each role; determine DP/TP/EP degrees; and apply role-specific core or memory settings subject to TTFT, TPOT, capacity, and power constraints. The resulting objective is a constrained Pareto optimization problem rather than simple power minimization.

A useful next step is to construct predictive models from the collected measurements. These models should capture computation, memory traffic, communication, synchronization, and idle power separately and estimate both latency and energy before deployment. Online feedback could then correct prediction error and adapt to workload drift. Extending the methodology to multi-tenant serving, heterogeneous accelerators, shared KV-cache tiers, node-level energy measurement, and carbon-aware placement would provide a more complete foundation for sustainable LLM infrastructure.

7 CONCLUSION

This paper establishes a systematic methodology for understanding the performance and GPU-energy behavior of non-disaggregated and disaggregated LLM serving. We compare Native, prefill–decode (PD), and attention–FFN (AF) architectures in a common SGLang implementation under

matched models, request streams, SLOs, and accelerator budgets. The study spans dense and MoE models, diverse input/output regimes, DP/TP/EP and hybrid parallelism, datacenter and consumer GPUs, NVLink/PCIe/RDMA communication, compute-memory capability, and GPU operating settings. By measuring latency, throughput, power, and total energy over the same request interval, the methodology avoids treating lower power as synonymous with greater energy efficiency.

The central premise of this work is that serving architecture cannot be evaluated independently of workload, placement, communication, and hardware. Disaggregation offers specialization and independent scaling, but its benefits must offset state transfer, activation communication, synchronization, and additional device residency. Likewise, a throughput-optimal parallel plan or a lower-frequency operating point is not necessarily energy optimal. Reproducible deployment manifests, communication validation, SLO-aware filtering, per-GPU energy accounting, and repeated open-loop trials provide the evidence needed to identify these trade-offs.

Our characterization is intended to support both system designers and operators: it provides a common basis for selecting architectures and deployment configurations and motivates future controllers that jointly optimize architecture, parallelism, placement, and frequency. As LLM inference becomes a growing component of datacenter electricity demand, such architecture-aware performance-energy analysis will be essential for building scalable and sustainable serving systems.

ACKNOWLEDGMENTS

The authors would like to thank the anonymous reviewers for their constructive feedback.

REFERENCES

- [1] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 27 730–27 744.
- [2] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [4] M. D. Skarlinski, S. Cox, J. M. Laurent, D. Braza, M. Hinks, D. M. Hammerling, M. Choi, R. Arora *et al.*, "Paperqa: Retrieval-augmented generative agent for scientific research," *arXiv preprint arXiv:2312.07559*, 2024.
- [5] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha, "The AI scientist: Towards fully automated open-ended scientific discovery," *arXiv preprint arXiv:2408.06292*, 2024.
- [6] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu *et al.*, "Autogen: Enabling next-gen llm applications via multi-agent conversation," in *Proceedings of the Conference on Language Modeling*, 2024.
- [7] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation*, pp. 521–538, 2022.
- [8] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulanani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in llm inference with sarathi-serve," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 117–134.
- [9] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, and W. Lin, "Llumnix: Dynamic scheduling for large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 2024, pp. 173–191.
- [10] J. Duan, R. Lu, H. Duanmu, X. Li, X. Zhang, D. Lin, I. Stoica, and H. Zhang, "Muxserve: Flexible spatial-temporal multiplexing for multiple llm serving," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024.
- [11] K. Hong, X. Li, L. Chen, Q. Mao, X. Ning, G. Dai, S. Yan, Y. Liang, and Y. Wang, "Sola: Optimizing slo attainment for large language model serving with state-aware scheduling," in *Proceedings of the 8th Annual Conference on Machine Learning and Systems*, 2025.
- [12] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, pp. 611–626, 2023.
- [13] L. Zheng, L. Yin, Z. Xie, J. H. Sun, C. H. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez *et al.*, "Sglang: Efficient execution of structured language model programs," *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [14] R. Qin, Z. Li, W. He, M. Wu, Y. Zhang *et al.*, "Mooncake: A kvcache-centric disaggregated architecture for llm serving," in *Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST 25)*, 2025.
- [15] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Re, C. Barrett *et al.*, "H2o: Heavy-hitter oracle for efficient generative inference of large language models," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 34 661–34 710.
- [16] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *The Twelfth International Conference on Learning Representations*, 2024.
- [17] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, "Snapkv: Llm knows what you are looking for before generation," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 22 947–22 970.
- [18] R. Chen, Z. Wang, B. Cao, T. Wu, S. Zheng, X. Li, X. Wei, S. Yan, M. Li, and Y. Liang, "Arkval: Efficient generative llm inference with recallable key-value eviction," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [19] W. Lee, J. Lee, J. Seo, and J. Sim, "Infinigen: Efficient generative inference of large language models with dynamic kv cache management," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 2024, pp. 155–172.
- [20] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan *et al.*, "Cachegen: Kv cache compression and streaming for fast large language model serving," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024.
- [21] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, "Qserve: W4a8kv4 quantization and system co-design for efficient llm serving," in *Proceedings of the 8th Annual Conference on Machine Learning and Systems*, 2025.
- [22] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," *arXiv preprint arXiv:1909.08053*, 2019.
- [23] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He, "DeepSpeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale," in *Proceedings of the 39th International Conference on Machine Learning*, 2022, pp. 18 332–18 346.
- [24] H. Liu, M. Zaharia, and P. Abbeel, "Ring attention with blockwise transformers for near-infinite context," in *The Twelfth International Conference on Learning Representations*, 2024.
- [25] B. Wu, S. Liu, Y. Zhong, P. Sun, X. Liu, and X. Jin, "Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, 2024, pp. 640–654.

- [26] Q. Su, W. Zhao, X. Li, M. Andoorveedu, C. Jiang, Z. Zhu, K. Song, C. Giannoula, and G. Pekhimenko, "Seesaw: High-throughput llm inference via model re-sharding," in *Proceedings of the 8th Annual Conference on Machine Learning and Systems*, 2025.
- [27] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "{DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX symposium on operating systems design and implementation (OSDI 24)*, 2024, pp. 193–210.
- [28] R. Zhu, Z. Jiang, C. Jin, P. Wu, C. A. Stuardo, D. Wang, X. Zhang, H. Zhou, H. Wei, Y. Cheng *et al.*, "Megascall-infer: Efficient mixture-of-experts model serving with disaggregated expert parallelism," in *Proceedings of the ACM SIGCOMM 2025 Conference*, 2025, pp. 592–608.
- [29] P. Patel, E. Choukse, C. Zhang, A. Shah, I. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative llm inference using phase splitting," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 118–132.
- [30] F. Strati, S. McAllister, A. Phanishayee, J. Tarnawski, and A. Klimovic, "Dejavu: Kv-cache streaming for fast, fault-tolerant generative llm serving," in *Proceedings of the 41st International Conference on Machine Learning*, ser. *Proceedings of Machine Learning Research*, vol. 235, 2024.
- [31] B. Lin, C. Zhang, T. Peng, H. Zhao, W. Xiao, M. Sun, A. Liu, Z. Zhang, L. Li, X. Qiu *et al.*, "Infinite-llm: Efficient llm service for long context with distattention and distributed kvcache," *arXiv preprint arXiv:2401.02669*, 2024.
- [32] International Energy Agency, "Energy and ai," International Energy Agency, Paris, Tech. Rep., 2025. [Online]. Available: <https://www.iea.org/reports/energy-and-ai>
- [33] Z. Zhang, F. Strati, and A. Klimovic, "Characterizing energy and performance for distributed training of large language models," in *Proceedings of the 6th Workshop on Machine Learning and Systems (EuroMLSys '26)*. ACM, 2026, pp. 208–214.
- [34] M. M. A. Ifath and I. Haque, "Characterizing performance–energy trade-offs of large language models in multi-request workflows," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 10, no. 1, pp. 1–26, 2026.
- [35] NVIDIA, "NVIDIA A800 Tensor Core GPU Datasheet," <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a800/nvidia-a800-datasheet-nvidia-us-2540751-web.pdf>, 2022.
- [36] —, "NVIDIA L40S GPU Specifications," <https://www.nvidia.com/en-us/data-center/l40s/>, 2023.
- [37] —, "NVIDIA RTX Blackwell GPU Architecture," <https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf>, 2025.
- [38] —, "NVIDIA L20 GPU Product Specification," <https://www.nvidia.com/>, 2024.
- [39] P. Vellaisamy, V. Lam, S. Blanton, and J. P. Shen, "Characterization of request and token energy costs for llm inference workloads on gpu platforms," *arXiv preprint arXiv:2608.28044*, 2026.
- [40] J.-W. Chung, J. J. Ma, R. Wu, J. Liu, O. J. Kweon, Y. Xia, Z. Wu, and M. Chowdhury, "The ml.energy benchmark: Toward automated inference energy measurement and optimization," in *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2025.
- [41] J. Stojkovic, C. Zhang, Í. Goiri, J. Torrellas, and E. Choukse, "Dynamollm: Designing llm inference clusters for performance and energy efficiency," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1348–1362.
- [42] A. K. Kakolyris, D. Masouros, P. Vavaroutsos, S. Xydis, and D. Soudris, "throttll'em: Predictive gpu throttling for energy efficient llm inference serving," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1363–1378.
- [43] C. Hankendi, R. Shahout, M. Yu, and A. K. Coskun, "Pals: Power-aware llm serving for mixture-of-experts models," *arXiv preprint arXiv:2605.21427*, 2026.
- [44] Y. Jiang, F. Fu, X. Yao, T. Wang, B. Cui, A. Klimovic, and E. Yoneki, "Thunderserve: High-performance and cost-efficient llm serving in cloud environments," in *Proceedings of the 8th Annual Conference on Machine Learning and Systems*, 2025.
- [45] Y. Mei, Y. Zhuang, X. Miao, J. Yang, Z. Jia, and R. Vinayak, "Helix: Serving large language models over heterogeneous gpus and network via max-flow," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025.
- [46] Q. Liu, D. Huang, M. Zapater, and D. Atienza, "Greenllm: SLo-aware dynamic frequency scaling for energy-efficient llm serving," *arXiv preprint arXiv:2508.16449*, 2025.
- [47] O. Basit, Y. Liu, Z. J. Kong, and Y. C. Hu, "Biscale: Energy-efficient disaggregated llm serving via phase-aware placement and dvfs," *arXiv e-prints*, pp. arXiv–2602, 2026.
- [48] C. Hu, L. Xu, T. Liu, M. Lyu, Y. Li, S. Wang, S. Quan, Y. Yang, W. Tang, Y. Wang *et al.*, "Energy-efficient llm serving via disaggregated attention–ffn and flexible frequency scaling," *arXiv preprint arXiv:2608.01891*, 2026.
- [49] Z. Wang, Y. Zhang, F. Wei, B. Wang, Y. Liu, Z. Hu, J. Zhang, X. Xu, J. He, X. Wang *et al.*, "Using analytical performance/power model and fine-grained dvfs to enhance ai accelerator energy efficiency," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025, pp. 1118–1132.
- [50] A. Agrawal, N. Kedia, J. Mohan, A. Panwar, N. Kwatra, B. S. Gulavani, R. Ramjee, and A. Tumanov, "Vidur: A large-scale simulation framework for llm inference," in *Proceedings of the 7th Annual Conference on Machine Learning and Systems*, 2024.
- [51] X. Cui, C.-J. M. Liang, Z. Tang, J. Xing, and H. Qiu, "Opscale: Operator-level provisioning and autoscaling for llm serving," *arXiv preprint arXiv:2608.13499*, 2026.
- [52] R. Pan, Z. Wang, Z. Jia, C. Karakus, L. Zancato, T. Dao, Y. Wang, and R. Netravali, "Marconi: Prefix caching for the era of hybrid llms," in *Proceedings of the 8th Annual Conference on Machine Learning and Systems*, 2025.
- [53] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, "Cacheblend: Fast large language model serving for rag with cached knowledge fusion," in *Proceedings of the Twentieth European Conference on Computer Systems*, 2025.
- [54] S. Yang, J. Guo, H. Tang, Q. Hu, G. Xiao, J. Tang, Y. Lin, Z. Liu, Y. Lu, and S. Han, "Lserve: Efficient long-sequence llm serving with unified sparse attention," in *Proceedings of the 8th Annual Conference on Machine Learning and Systems*, 2025.
- [55] W. Gao, X. Zhou, P. Sun, T. Zhang, and Y. Wen, "Rethinking key-value cache compression techniques for large language model serving," in *Proceedings of the 8th Annual Conference on Machine Learning and Systems*, 2025.
- [56] A. V. Kumar, G. Antichi, and R. Singh, "Aqua: Network-accelerated memory offloading for llms in scale-up gpu domains," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025.
- [57] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, "Flashinfer: Efficient and customizable attention engine for llm inference serving," in *Proceedings of the 8th Annual Conference on Machine Learning and Systems*, 2025.